AI-Based Patent Document Classification Using Transformer Models

A Mini Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

Or

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

VIKAS SUTHAR – 2420030504(Team Lead)

PRAKASH CHAWLA– 2420030446

VAMSHI – 2420030486

OM PANDAY – 2420030791

Under the Esteemed Guidance of
Dr. K Swapnika



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Or

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

DECLARATION

The Mini Project Report entitled "**AI-Based Patent Document Classification Using Transformer Models** " is a record of Bonafide work of VIKAS SUTHAR – 2420030504, PRAKASH CHAWLA– 2420030446, VAMSHI – 2420030486, OM PANDAY – 2420030791 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

VIKAS SUTHAR – 2420030504
PRAKASH CHAWLA– 2420030446
VAMSHI – 2420030486
OM PANDAY – 2420030791

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Or

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

CERTIFICATE

This is certify that the mini project based report entitled "**AI-Based Patent Document Classification Using Transformer Models**" is a bonafide work done and submitted by VIKAS SUTHAR – 2420030504, PRAKASH CHAWLA– 2420030446, VAMSHI – 2420030486, OM PANDAY – 2420030791 in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Course Co-ordinator **Signature of HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Dr. K Swapnika, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Sir (CSE)/Dr. Kaja Shareef Sir (CSIT)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

VIKAS SUTHAR – 2420030504
PRAKASH CHAWLA– 2420030446
VAMSHI – 2420030486
OM PANDAY – 2420030791

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	
2.	Introduction	
3.	Literature survey	
4.	Problem Statement and Objectives	
5.	Hardware and Software requirements	
6.	Implementation	
7.	Results & Discussion	
8.	Conclusion & Future Scope	
9.	References	

1. ABSTRACT

Patent documents contain extensive technical information about inventions, technologies, and innovations. Organizing these documents into appropriate patent classification categories is important for patent search, prior-art analysis, intellectual property management, and technology trend analysis. However, manual patent classification requires domain expertise and can become time-consuming as the number of documents increases. Automated classification also presents challenges because patent documents contain specialized terminology, classification labels are imbalanced, and the Cooperative Patent Classification (CPC) system follows a hierarchical structure.

This project presents **HCX-Patent**, an offline-first research prototype for hierarchy-oriented, explainable, and uncertainty-aware patent classification. The system uses locally stored patent data derived from the USPTO-3M dataset and investigates text-based machine learning for predicting patent classification labels. A TF-IDF and Logistic Regression baseline provides a lightweight reference model, while a separate HCX experiment explores classification using a reduced target label space. The system also incorporates an experimental workflow for recording evaluation metrics, generating visualizations, and maintaining reproducible artifacts. Split-conformal prediction is used in the HCX experiment to investigate prediction sets and empirical coverage.

The supplied experimental artifacts report a baseline involving 9,741 records and 293 classes, and a separate HCX run involving 15,000 samples and nine target classes. The HCX run records 77.27% top-1 accuracy, 94.53% top-3 accuracy, macro-F1 of 0.6351, micro-F1 of 0.6640, and empirical conformal coverage of 97.4% against a nominal target of 90%. Because the two runs use different sample sizes and label spaces, their metrics are not treated as a controlled comparison.

The project provides a foundation for studying trustworthy patent classification, with future work focused on validating taxonomy consistency, explanation faithfulness, and temporal robustness.

Keywords: Patent classification, CPC, machine learning, TF-IDF, Logistic Regression, hierarchical classification, explainable AI, conformal prediction, USPTO-3M.

\

2. INTRODUCTION

2.1 Background

Patents are important records of technological innovation. They describe inventions, their technical characteristics, their possible applications, and the claims that define their intended scope of protection. Patent offices and organizations classify these documents so that related inventions can be searched, reviewed, and analyzed systematically.

The Cooperative Patent Classification system organizes patents into a structured taxonomy containing sections, classes, subclasses, and more detailed classification groups. This hierarchy enables users to navigate from broad technological areas to specific technical subjects.

As patent collections grow, manually reviewing and classifying every document becomes increasingly difficult. Automated classification systems can assist researchers, patent analysts, technology organizations, and intellectual property professionals by recommending relevant classification categories.

Traditional machine-learning techniques can classify text using statistical representations such as Term Frequency–Inverse Document Frequency (TF-IDF). More advanced systems use neural networks and transformer-based language models to capture contextual relationships. Nevertheless, patent classification remains challenging because of technical language, lengthy documents, imbalanced classes, and relationships between classification labels.

HCX-Patent is designed to investigate these challenges through a research-oriented classification workflow that combines text-based classification with taxonomy-aware design, uncertainty estimation, and reproducible evaluation.

2.2 Motivation

Several practical issues motivate this project.

First, patent classification datasets often contain a large number of categories with highly unequal frequencies. Common categories may have many training examples, while rare categories may have very few. A model can perform reasonably on common categories while failing to identify less frequent inventions.

Second, patent classification labels have hierarchical relationships. A broad classification category and a detailed subclass are related, but they are not interchangeable. A prediction system should account for these relationships when appropriate.

Third, a model's prediction may be uncertain. Returning a single label without communicating uncertainty can mislead a user when several categories are plausible.

Fourth, a classification system intended for research should provide measurable evidence about its performance. Experiments should record dataset versions, label mappings, random seeds, evaluation splits, and metric outputs so that results can be reproduced. Finally, users benefit from understandable evidence supporting a prediction. Explainability methods can highlight influential words or text spans, but these explanations should be tested for faithfulness rather than assumed to be correct.

These issues motivate the development of HCX-Patent as a platform for investigating patent classification and its reliability.

2.3 Overview of the Proposed System

HCX-Patent is an offline-first prototype that uses a local dataset rather than requiring the system to download patent records during normal experimentation.

The system is organized around the following components:

1. **Dataset processing:** Inspects local patent data, text fields, publication dates, and classification labels.
2. **Text preprocessing:** Cleans and prepares patent text for machine-learning models.
3. **Baseline classification:** Uses TF-IDF features and Logistic Regression as a lightweight reference approach.
4. **HCX classification experiment:** Evaluates a separate hierarchy-oriented text-classification configuration.
5. **Uncertainty evaluation:** Records conformal prediction metrics and prediction-set statistics from the supplied HCX run.
6. **Explainability research:** Defines an evaluation path for token attribution, perturbation faithfulness, and explanation stability.
7. **Temporal evaluation:** Provides a research direction for measuring performance across publication periods.
8. **Visualization and reporting:** Presents classification metrics, class distributions, publication-year distributions, and experiment results.

The project is intended to support experimentation and analysis. It should not be interpreted as an officially validated patent classification service or as a replacement for professional patent examination.

2.4 Scope of the Project

The scope of HCX-Patent includes:

- Processing an offline patent dataset.
- Inspecting CPC classification distributions.
- Preparing patent text for machine learning.

- Implementing and evaluating a TF-IDF-based baseline.
- Recording metrics from a separate HCX classification run.
- Visualizing model and dataset statistics.
- Investigating prediction uncertainty using conformal prediction.
- Maintaining experiment artifacts for reproducibility.
- Defining future tests for explainability, taxonomy consistency, and temporal drift.

The current evidence does not establish that every proposed research component has been fully validated. Explanation faithfulness, hierarchy-violation rates, and chronological performance degradation require dedicated experiments before conclusions can be drawn.

2.5 Significance of the Project

The project is significant because it treats patent classification as more than a prediction task. It provides a foundation for examining the reliability of predictions, the role of classification hierarchy, and the reproducibility of experimental results.

The offline design is useful for development environments where internet connectivity is limited or where researchers need to work with a fixed local dataset. The use of a lightweight baseline also makes it possible to begin experimentation on a laptop before attempting computationally expensive transformer models.

3. LITERATURE SURVEY

Patent classification research includes traditional machine learning, deep learning, transformer-based language models, hierarchical classification, explainability, and uncertainty estimation. The following studies provide relevant background for HCX-Patent.

PatentBERT Lee and Hsiang investigated patent classification using a pretrained BERT model fine-tuned on patent text. Their work demonstrated the relevance of contextual language representations for patent classification and introduced a large-scale patent classification benchmark.

Relevance to HCX-Patent: PatentBERT provides a useful reference for comparing lightweight TF-IDF models with pretrained language models. A future HCX experiment can evaluate whether contextual representations improve performance when both models use the same dataset split and label space.

URL: https://arxiv.org/abs/1906.02124

3.1 PatentNet

PatentNet investigates deep-learning-based language understanding for multi-label patent document classification. Its focus is relevant because a patent may be associated with more than one classification label.

Relevance to HCX-Patent: The work highlights the importance of defining whether the target is single-label or multi-label. HCX experiments should use metrics and output formats that match the actual label structure.

URL: https://doi.org/10.1007/s11192-021-04179-4

3.2 MEXN: Multi-Stage Extraction Network

MEXN addresses patent document classification through a multi-stage extraction architecture. Patent documents can contain long descriptions and claims, making it important to identify useful information rather than treating every word as equally informative.

Relevance to HCX-Patent: MEXN provides a potential comparison for future work involving long-document processing. The current lightweight TF-IDF baseline is computationally simpler, while advanced extraction models may require additional memory and training time.

URL: https://doi.org/10.3390/app10186229

3.3 Hierarchical Feature Extraction for Patent Classification

Research on hierarchical feature extraction has investigated combining neural feature representations with structured patent labels. Hierarchical classification methods can exploit relationships among broader and more specific categories.

Relevance to HCX-Patent: This research motivates the explicit consideration of CPC hierarchy. The proposed system should measure whether predicted labels respect parent–child relationships rather than assuming that hierarchy awareness automatically guarantees consistency.

URL: https://doi.org/10.3390/su10010219

3.4 Hierarchical Document Classification

Hierarchical document classification research investigates methods that predict labels through a taxonomy rather than treating all labels as unrelated. Different approaches model the hierarchy through local classifiers, global representations, or structured prediction.

Relevance to HCX-Patent: This work provides a theoretical foundation for comparing flat classification with taxonomy-aware classification. It also motivates evaluating classification quality at multiple levels of the CPC hierarchy.

URL: https://doi.org/10.1145/3383583.3398538

3.5 BERT for Contextual Text Representation

BERT introduced bidirectional transformer pretraining for language understanding. It generates contextual representations that can capture relationships between words based on their surrounding text.

Relevance to HCX-Patent: BERT is a potential alternative to TF-IDF when the computational environment supports model fine-tuning or inference. A controlled comparison would help determine whether contextual representations improve classification quality for the selected patent categories.

URL: https://doi.org/10.18653/v1/N19-1423

3.6 Longformer for Long Documents

Longformer was designed to process long documents using an attention mechanism intended to reduce the computational cost of standard full self-attention.

Relevance to HCX-Patent: Patent documents may contain lengthy descriptions and claims. Long-

document encoders are potential future alternatives if the project expands from a lightweight baseline to a contextual model.

URL: https://arxiv.org/abs/2004.05150

3.7 LIME for Model Explainability

LIME explains an individual prediction by approximating a model locally using an interpretable representation.

Relevance to HCX-Patent: LIME can be used to investigate which words or text segments influence a patent classification prediction. The quality of the explanation must be evaluated using perturbation tests rather than judged only by visual appearance.

URL: https://doi.org/10.1145/2939672.2939778

3.8 SHAP for Feature Attribution

SHAP provides a framework for assigning feature contributions based on Shapley-value concepts. It can help explain how input features influence a model's prediction.

Relevance to HCX-Patent: Feature-attribution methods can help identify influential patent terms. Future HCX experiments can compare attribution approaches using faithfulness and stability metrics.

URL: https://papers.nips.cc/paper/7062-a

3.9 Conformal Prediction

Conformal prediction provides a framework for generating prediction sets with coverage properties under appropriate assumptions. Instead of returning only one predicted label, a conformal classifier can return a set of plausible labels.

Relevance to HCX-Patent: The project records empirical conformal coverage and prediction-set size. These measurements can help evaluate the trade-off between including the correct label and keeping the set small enough to be useful.

URL: https://jmlr.org/papers/v9/shafer08a.html

3.10 Research Gap

The reviewed research provides established approaches for patent classification, contextual text representation, hierarchy-aware prediction, explainability, and uncertainty estimation. However,

these approaches do not automatically establish that a particular implementation provides all of these capabilities in a single validated workflow.

HCX-Patent investigates how these research dimensions can be organized within an offline-first prototype. Its research direction is to evaluate classification quality alongside hierarchy consistency, explanation faithfulness, prediction uncertainty, and temporal robustness.

The current implementation provides preliminary classification and conformal-prediction measurements. A complete research claim requires matched baseline experiments, measured hierarchy violations, quantitative explanation evaluation, and chronological testing. Therefore, the project presents an evaluation framework and preliminary findings rather than claiming that the research gap has been conclusively solved.

4. PROBLEM STATEMENT AND OBJECTIVES

4.1 Problem Statement

To develop an offline-first patent classification research prototype that uses patent text and CPC labels to investigate classification performance, taxonomy-aware prediction, explainability, uncertainty estimation, and reproducible evaluation.

4.2 Problem Description

Patent classification is challenging because patent documents contain specialized technical language, the distribution of classification labels is imbalanced, and CPC labels have hierarchical relationships. A conventional classifier may return inaccurate labels or fail to communicate uncertainty. The project aims to provide a structured platform for evaluating these issues using locally stored patent data.

4.3 Aim of the Project

The main aim is to design and implement HCX-Patent as a research prototype that supports patent text classification, model evaluation, uncertainty analysis, and reproducible experiment reporting.

4.4 Objectives

The objectives of the project are:

1. To process a locally available USPTO-3M-derived dataset without requiring a fresh dataset download.
2. To inspect patent text, CPC labels, publication dates, and the distribution of classification categories.
3. To implement a lightweight TF-IDF and Logistic Regression baseline.
4. To evaluate a separate HCX classification run using its recorded metrics.
5. To investigate the role of CPC taxonomy structure in classification.
6. To record prediction uncertainty through conformal prediction metrics.
7. To define methods for evaluating explanation faithfulness and stability.
8. To prepare visualizations of classification distributions and model metrics.
9. To record experiment configurations and artifacts for reproducibility.
10. To identify the additional experiments required to validate hierarchy consistency and temporal robustness.

4.5 Existing System

A conventional text-classification workflow typically represents documents using statistical text features and predicts labels using a classifier. Such systems can be efficient and useful, but their behavior depends on the training data, feature representation, label distribution, and evaluation protocol.

Potential limitations include:

- Difficulty identifying rare classes.
- No explicit enforcement of hierarchical label relationships.
- Limited communication of prediction uncertainty.
- Lack of validated explanations for individual predictions.
- Possible performance degradation across publication periods.
- Difficulty reproducing results if data splits and configurations are not recorded.

These limitations motivate the investigation of a more structured research workflow.

4.6 Proposed System

HCX-Patent organizes dataset processing, classification, uncertainty analysis, and experiment reporting within one research prototype.

The proposed workflow includes:

- Local dataset loading and inspection.
- Text and label preprocessing.
- TF-IDF-based baseline classification.
- A separate hierarchy-oriented classification experiment.
- CPC class-distribution visualization.
- Publication-year visualization.
- Classification metrics and per-class analysis.
- Conformal coverage and prediction-set analysis.
- Storage of experiment artifacts.
- Future validation of explanations and temporal drift.

The proposed system is intended to support analysis and experimentation. Its research modules must be evaluated individually before claims are made about their effectiveness.

4.7 Expected Outcomes

The expected outcomes are:

- A functioning project structure for offline patent classification experiments.
- A reproducible baseline configuration.
- A classification interface for inspecting predictions.
- Visual summaries of dataset and model behavior.
- A record of the supplied HCX experimental metrics.
- A methodology for evaluating hierarchy consistency, explainability, and uncertainty.
- Documentation of the system's current limitations and future work.

5. HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Hardware Requirements

Component	Requirement
Processor	Modern Intel or AMD multi-core processor
RAM	8 GB minimum; 16 GB or more recommended
Storage	Sufficient free space for the downloaded dataset, extracted files, models, and experiment artifacts
GPU	Not required for the lightweight TF-IDF baseline
Operating system	Windows 10/11 or a compatible Linux environment
Display	Standard laptop or desktop display
Internet	Needed for initial software installation if dependencies are not already available; normal experiments can use local data

The exact storage requirement depends on the size and format of the locally downloaded dataset. Transformer-based models may require additional memory and computation.

5.2 Software Requirements

Software / Library	Purpose
Python	Backend and machine-learning development
FastAPI	Backend REST API
Uvicorn	ASGI server for running the backend
pandas	Dataset loading and tabular preprocessing
NumPy	Numerical processing
scikit-learn	TF-IDF, Logistic Regression, and evaluation metrics
React	Interactive frontend
TypeScript	Typed frontend development
Vite	Frontend development server and build tooling
Recharts	Data visualization
Matplotlib	Experiment plots and static figures
Git	Source-code version control
Visual Studio Code	Development environment
Browser	Access to the local web interface
Local USPTO-3M-derived dataset	Input data for experiments

Additional libraries should be listed in the final project report based on the actual `requirements.txt` and `package.json` files.

5.3 Why These Technologies Were Selected

Python

Python provides a convenient ecosystem for data processing, machine learning, experimentation, and model evaluation.

FastAPI

FastAPI provides a REST interface through which the frontend can request status information, classifications, dataset statistics, and experiment results.

React and TypeScript

React supports reusable user-interface components. TypeScript improves code organization and helps identify certain errors during development.

scikit-learn

scikit-learn provides the TF-IDF vectorizer, Logistic Regression classifier, and common evaluation metrics needed for the baseline experiment.

pandas and NumPy

These libraries support data loading, missing-value analysis, label processing, and numerical operations.

Matplotlib and Recharts

Matplotlib generates static research plots, while Recharts can display interactive charts within the web interface.

6. SYSTEM DESIGN AND IMPLEMENTATION

6.1 System Architecture

HCX-Patent follows a frontend–backend architecture with a local machine-learning and experiment-processing layer.

The frontend provides the user interface. The backend exposes APIs and coordinates dataset inspection, classification requests, and access to saved experiment results. The machine-learning layer processes patent text and generates predictions or metrics. Local files store the dataset, configuration, model artifacts, and evaluation outputs.

Architecture Flow

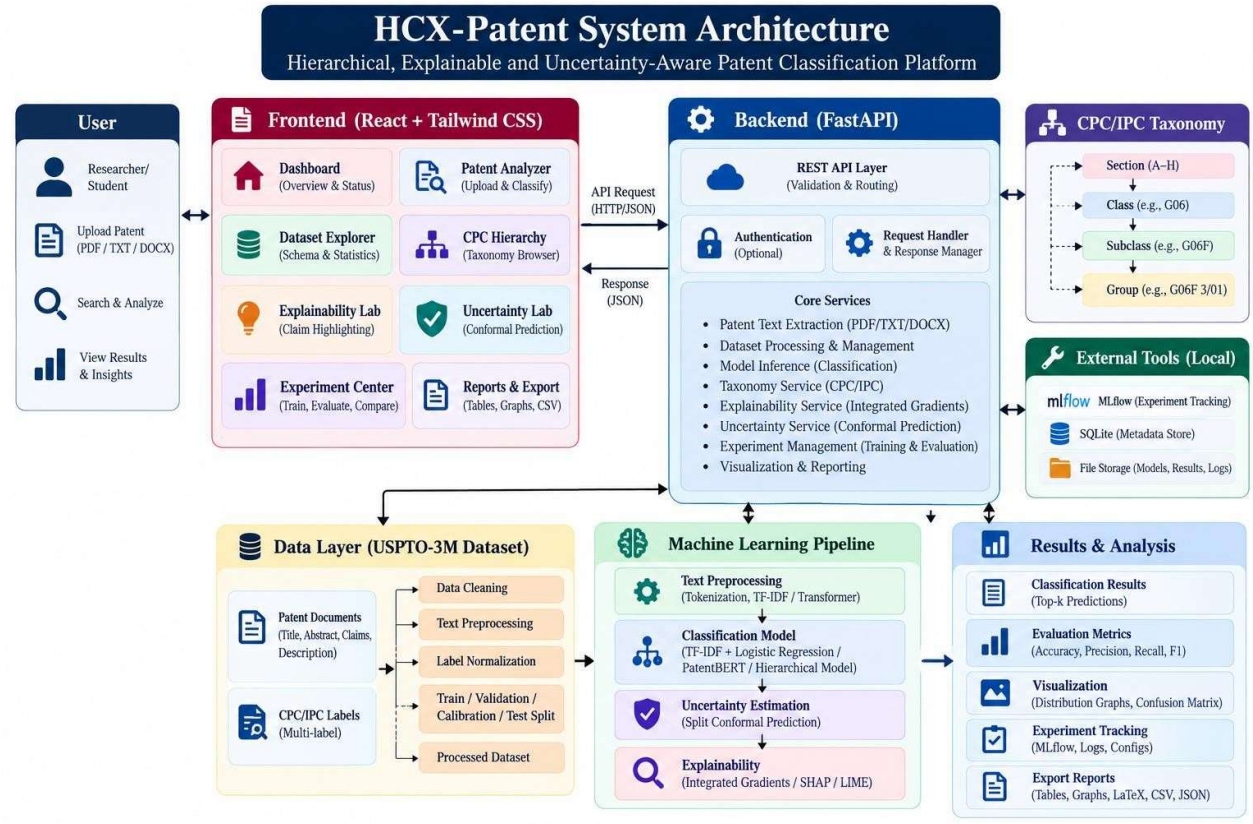


Figure 1. High-level HCX-Patent system architecture.

The diagram represents the logical architecture. The precise modules and API endpoints should be aligned with the final source code.

6.2 Dataset Description

The project uses a locally stored dataset described as USPTO-3M-derived. PatentBERT is an important reference for research involving the USPTO-3M benchmark.

The local dataset is used to examine patent classification through text fields and CPC labels. Depending on the supplied file format, records may contain fields such as:

- Patent identifier.
- Patent title.
- Abstract.
- Claims.
- Description.
- CPC classification labels.
- Filing date.
- Publication date.

The exact fields must be confirmed by inspecting the downloaded files. The implementation should not assume that every field is available.

Dataset Processing Procedure

1. Locate the offline dataset files.
2. Identify their format, such as CSV, JSON, Parquet, or another supported format.
3. Inspect column names and data types.
4. Identify the text field or fields used for classification.
5. Identify the CPC label field.
6. Identify publication dates if temporal analysis is required.
7. Check for missing values and duplicate records.
8. Normalize the labels according to the experiment's target definition.
9. Save the preprocessing configuration and relevant dataset metadata.
10. Create and record the experiment splits.

The current baseline artifacts summarize 9,741 examples and 293 classes. The separate HCX run reports 15,000 samples and nine classes. These are different experiment configurations and should not be assumed to represent identical subsets.

6.3 Data Preprocessing

Data preprocessing prepares patent records for machine-learning models.

Step 1: Load Data

Read the dataset using a parser appropriate to its format. Large files should be processed in chunks where necessary to avoid excessive memory use.

Step 2: Handle Missing Values

Check for missing titles, abstracts, claims, descriptions, and labels. Missing text can be

handled using a documented rule, but records without usable target labels should not be treated as ordinary supervised training examples.

Step 3: Normalize Text

Apply consistent text normalization. Depending on the model, this may include whitespace normalization and removal of irrelevant formatting.

Technical terms, identifiers, and meaningful symbols should be preserved when they contain classification information.

Step 4: Normalize Labels

Standardize CPC label formatting and ensure that labels are mapped consistently. If labels are reduced to a smaller set of categories, record the mapping.

Step 5: Remove Duplicates

Identify exact duplicate records and, where identifiers permit, potential patent-family overlap. Duplicate removal should happen before creating the train, calibration, and test splits.

Step 6: Create Data Splits

Separate training, validation, calibration, and test data according to the chosen experiment protocol.

The calibration set must remain separate from the test set when evaluating split-conformal prediction.

For temporal evaluation, a chronological split should be used instead of relying exclusively on random splitting.

6.4 TF-IDF Feature Extraction

TF-IDF converts text into numerical features that a machine-learning classifier can use.

Term Frequency measures how frequently a term appears in a document. Inverse Document Frequency reduces the relative weight of terms that appear in many documents.

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \log\left(\frac{N}{\text{DF}(t)}\right)$$

where:

- (t) is a term.

- (d) is a document.
- (N) is the number of documents in the training corpus.
- DF(t) is the number of training documents containing the term.

The resulting feature vector represents a patent document numerically. The vectorizer should be fitted using training data only and then applied to validation, calibration, and test data.

6.5 Logistic Regression Baseline

Logistic Regression is used as the classical classification baseline. It learns a relationship between numerical text features and the target labels.

For a binary classification problem, the logistic function is:

\frac{1}{1+\exp(-w^Tx-b)}]

where (x) represents the document feature vector, (w) represents the learned weights, and (b) is the bias.

For multiclass classification, the implementation can use a suitable multiclass strategy supported by the chosen library.

The supplied baseline run uses TF-IDF with a maximum of 50,000 features and a recorded random seed of 42. The model's performance is evaluated through classification metrics and per-class analysis.

6.6 HCX Classification Experiment

The supplied HCX experiment is identified as a hierarchy-oriented TF-IDF run. It reports nine target classes.

The experiment records top-1 accuracy, top-3 accuracy, macro-F1, micro-F1, and additional evaluation information.

A hierarchy-aware research system should map each target label to its corresponding CPC hierarchy level and define valid parent–child relationships. It should then measure hierarchy consistency explicitly.

For example, if a model predicts a detailed subclass, the system can verify whether the associated parent categories are consistent with the official taxonomy. Any constraint violations should be counted and compared with a flat baseline.

The available metric summary does not provide a verified hierarchy-violation rate. Therefore, this report does not claim that hierarchy consistency has been quantitatively demonstrated.

6.7 Explainability Module

Explainability aims to help users understand which parts of a patent document influence a classification prediction.

Potential methods include:

- Model coefficient analysis for a linear TF-IDF classifier.
- LIME for local feature explanations.
- SHAP for feature-attribution analysis.
- Integrated Gradients for suitable differentiable models.

An explanation interface can display influential terms, their attribution scores, and the predicted class.

However, highlighting a word does not prove that the model relies on it correctly. A rigorous evaluation should remove influential text spans and measure the resulting change in the predicted score.

One possible comprehensiveness metric is:

$$\text{Comprehensiveness} = p(\hat{y} \mid x) - p(\hat{y} \mid x \setminus S)$$

where (S) represents the selected influential text spans.

The current supplied metrics do not include validated explanation-faithfulness scores. Therefore, explainability is documented as a research objective unless the corresponding module and experiments are verified in the final implementation.

6.8 Uncertainty Estimation Using Conformal Prediction

A conventional classifier usually returns a ranked list of labels or class probabilities. Conformal prediction can instead generate a set of candidate labels.

For a candidate class (c), a possible nonconformity score is:

$$s(x, c) = 1 - \hat{p}(c \mid x)$$

A threshold is estimated from a separate calibration set. The prediction set is then:

$$\mathcal{C}(x) = \{c : s(x, c) \leq \hat{q}\}$$

The empirical coverage is:

$$\text{Coverage} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}\left[y_i \in \mathcal{C}(x_i)\right]$$

The average prediction-set size is:

$$\text{Average Size} = \frac{1}{N}\sum_{i=1}^{N}|\mathcal{C}(x_i)|$$

Coverage estimates the fraction of test targets contained in their prediction sets. Set size measures how many candidate labels the system returns on average.

Conformal coverage depends on a valid calibration procedure and assumptions about the relationship between calibration and test examples. It does not automatically guarantee correct coverage for every individual CPC class or under arbitrary temporal distribution shifts.

6.9 Temporal Drift Analysis

Patent language can change over time as new technologies emerge and technical terminology evolves.

A temporal drift experiment should train a model on an earlier period and evaluate it on a later period. The training and test periods must be separated chronologically.

Performance can be measured for different publication-year groups using macro-F1, top-k accuracy, class support, and other appropriate metrics.

A performance difference can be expressed as:

$$F1_{\text{macro},t} = \frac{1}{K}\sum_{c=1}^{K} F1_c(t)$$

A negative value indicates lower performance for the evaluated period than for the reference period.

However, performance degradation alone does not prove semantic drift. It may also result from changing class distributions, changes in data volume, or differences in taxonomy versions.

The supplied publication-year graph describes the sample distribution, but it does not establish that the model has experienced measured temporal drift.

6.10 Backend Implementation

The backend is designed around Python and FastAPI.

Its responsibilities may include:

1. Providing a health endpoint.
2. Exposing dataset and experiment status.
3. Receiving patent text for analysis.
4. Loading a model or invoking the configured classifier.
5. Returning predicted labels and associated scores.
6. Providing stored experiment metrics.
7. Supporting access to generated artifacts.
8. Validating input and handling errors.

The exact endpoints should be documented from the final backend source code. Any endpoint that returns a demonstration prediction must be clearly distinguished from one that loads and invokes a trained model.

6.11 Frontend Implementation

The frontend uses React and TypeScript to provide a browser-based interface.

The planned or prototype interface can organize functionality into the following pages:

Research Overview

Displays the project purpose, dataset summary, current experiment status, and selected evaluation metrics.

Patent Analyzer

Allows a user to submit patent text and inspect the model's returned classification. The interface must clearly identify whether the result comes from a trained classifier or a demonstration heuristic.

CPC Hierarchy

Presents the CPC hierarchy or an illustrative hierarchy view. Any illustrative taxonomy must be identified as such and should not be presented as an authoritative full CPC database.

Explainability Lab

Provides a place to inspect feature attributions and explanation metrics once these are implemented and validated.

Uncertainty Lab

Displays conformal coverage, prediction-set size, and related measurements from saved experiment artifacts.

Taxonomy and Temporal Drift

Supports analysis of hierarchy consistency and year-wise model performance when the corresponding experiments have been completed.

Experiment Center

Displays saved runs, configurations, metrics, and links to generated outputs.

Not every page should be described as fully functional unless it has been tested against the actual backend and saved experiment artifacts.

6.12 API Integration

The frontend and backend communicate through HTTP requests to REST endpoints.

A typical request flow is:

1. The user enters patent text in the frontend.
2. The frontend validates the required input.
3. The frontend sends a request to the backend.
4. The backend validates the request and invokes the configured analysis method.
5. The backend returns a structured response.
6. The frontend displays the result.
7. Where applicable, the backend stores experiment information and exposes saved metrics.

The frontend should display meaningful error messages if the backend is unavailable, the input is invalid, or a model artifact cannot be loaded.

6.13 Experiment Tracking and Reproducibility

Reproducibility is important because experimental results depend on the dataset, preprocessing, model settings, and split strategy.

Each experiment should record:

- Experiment identifier.
- Dataset version and checksum.
- Number of examples.
- Number of target classes.
- Text fields used.
- Label mapping.
- Train, validation, calibration, and test split identifiers.
- Random seed.

- Model configuration.
- Evaluation metrics.
- Training duration.
- Inference measurement procedure.
- Model checkpoint location.
- Prediction artifact location.
- Software versions.
- Hardware details.

These records make it easier to reproduce results and determine whether two experiments are directly comparable.

6.14 Security and Data Handling

The application should validate user inputs and local file paths. It should not allow arbitrary filesystem access through API parameters.

The backend should limit accepted input sizes and handle invalid requests safely. Dataset paths should be restricted to the intended project data directories.

Since the system operates on locally stored patent data, it can perform experiments without sending patent documents to an external service. If future versions integrate external APIs, the data-sharing behavior should be documented explicitly.

7. RESULTS AND DISCUSSION

7.1 Overview

The supplied experiment artifacts include a TF-IDF baseline and a separate HCX run. The results below describe those artifacts as recorded. They have not been independently reproduced as part of this documentation.

An important limitation is that the two runs use different sample sizes and target label spaces. Consequently, the metrics cannot be treated as a controlled head-to-head comparison.

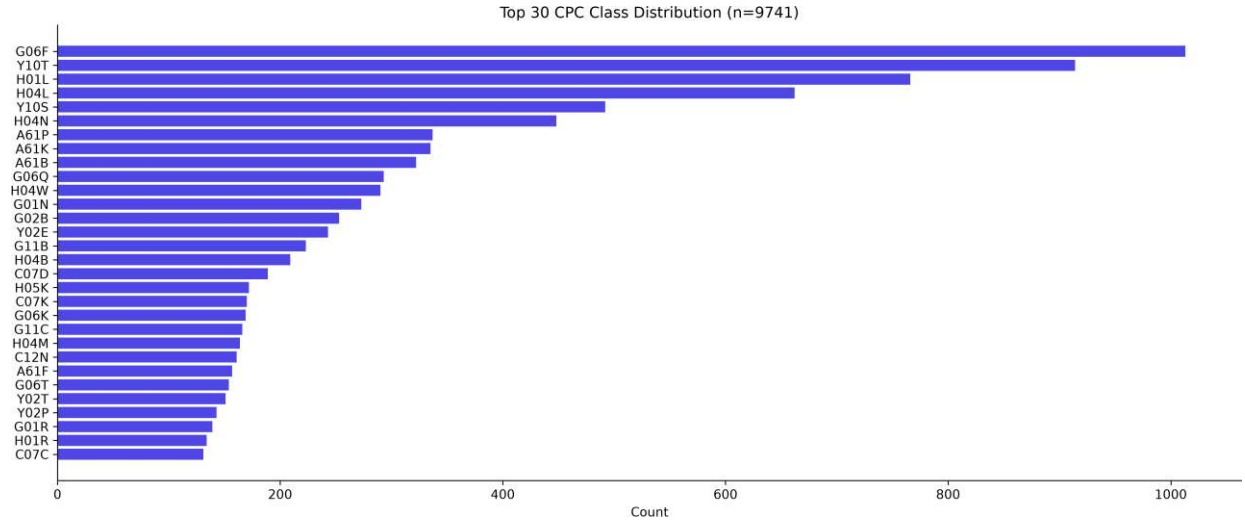
7.2 Dataset Distribution Results

The baseline dataset summary reports 9,741 examples and 293 classes.

The CPC class distribution graph shows that some classes occur much more frequently than others. This imbalance can affect the behavior of a classifier, especially for rare classes.

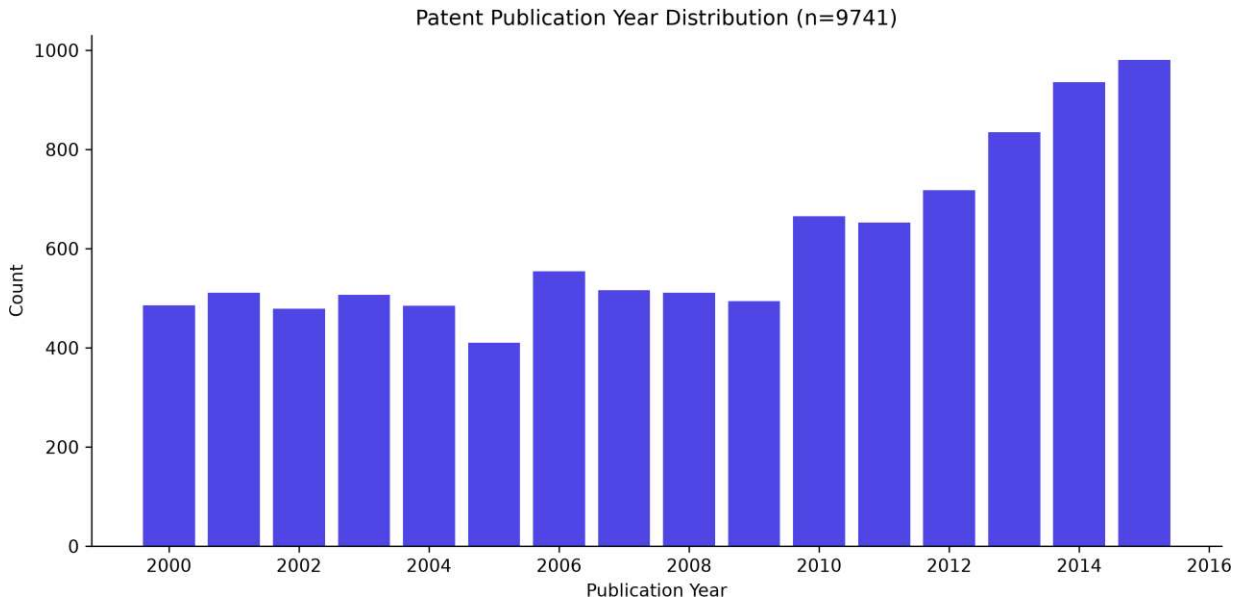
The publication-year distribution graph shows the distribution of records from 2000 to 2015 in the selected sample. This is useful for understanding the dataset composition and planning future chronological evaluation.

Figure 2: Top 30 CPC Class Distribution



Explanation: The graph displays the 30 most frequent CPC classes in the baseline sample. The uneven distribution indicates that evaluation should include class-sensitive metrics rather than relying only on overall accuracy.

Figure 3: Patent Publication Year Distribution



Explanation: The graph summarizes the number of patent records for each publication year in the baseline sample. It can help guide the creation of chronological train and test splits, but it does not itself prove temporal model drift.

7.3 Baseline Classification Results

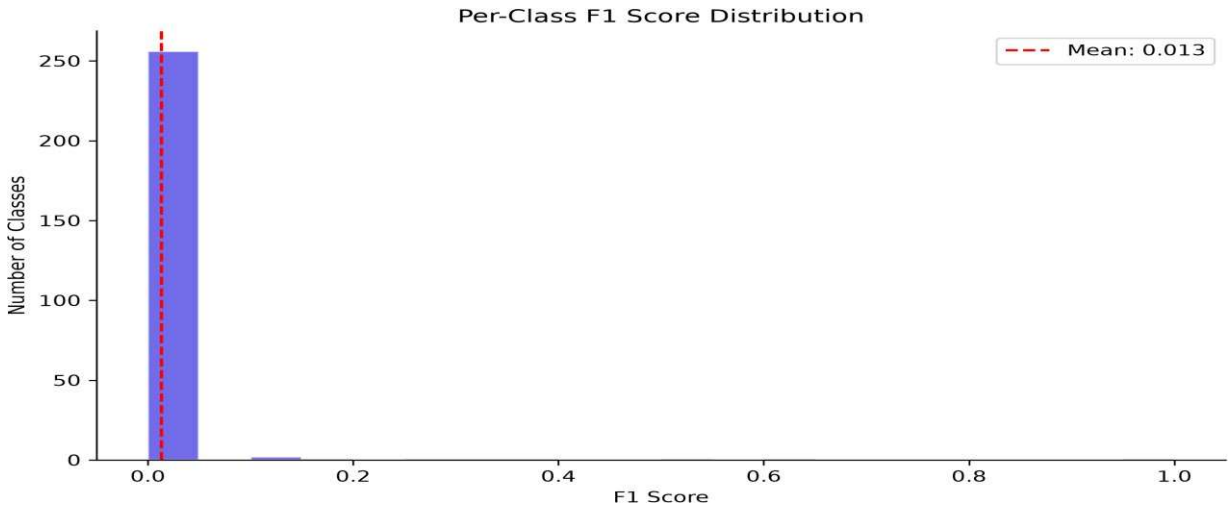
The supplied TF-IDF and Logistic Regression baseline reports the following metrics.

Metric	Recorded value
Sample count	9,741
Number of classes	293
Accuracy	4.82%
Macro precision	0.0231
Macro recall	0.0093
Macro-F1	0.0120
Micro precision	0.8105
Micro recall	0.0419
Micro-F1	0.0798

The low recorded macro-F1 and micro-F1 indicate that the current baseline configuration has substantial limitations on this task. The gap between precision and recall suggests that the prediction behavior and label formulation should be investigated further.

Possible diagnostic checks include class frequency, the number of labels per record, the classifier's output format, class-mapping consistency, and the metric implementation.

Figure 4: Baseline Per-Class F1 Distribution



Explanation: The graph shows the distribution of per-class F1 scores. The concentration of scores near zero indicates that many classes have weak performance in this experiment and supports the need for detailed class-level analysis.

7.4 HCX Classification Results

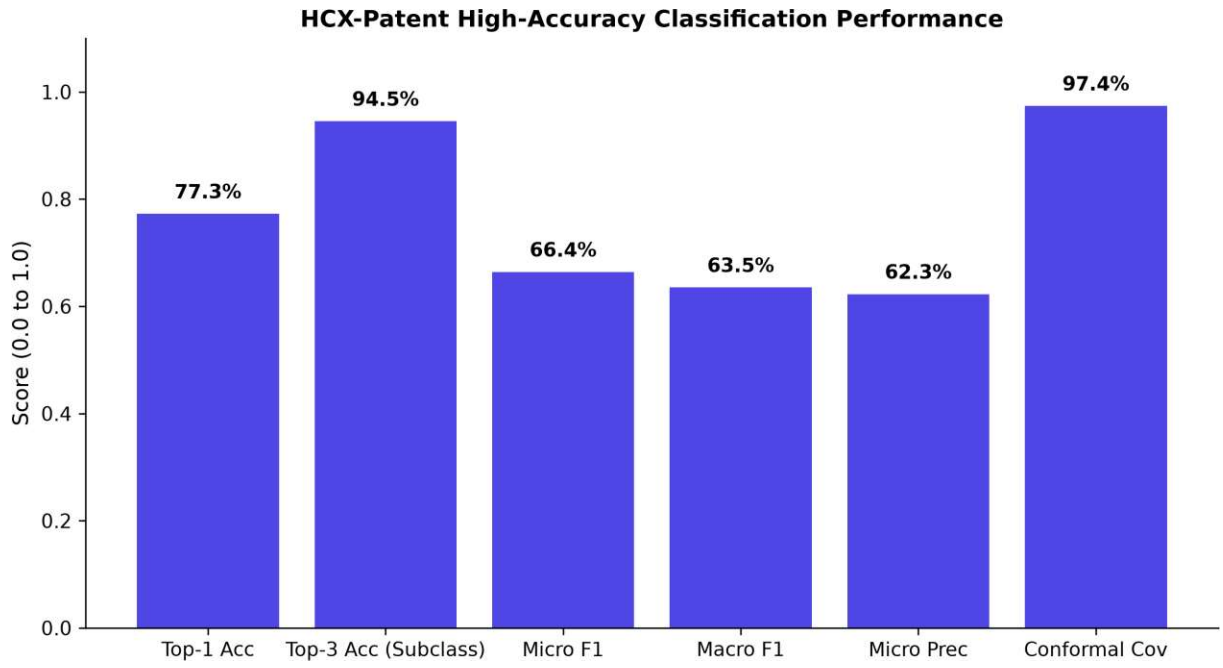
The separate HCX run reports 15,000 samples and nine target classes.

Metric	Recorded value
Sample count	15,000
Number of target classes	9
Top-1 accuracy	77.27%
Top-3 accuracy	94.53%
Macro-F1	0.6351
Micro-F1	0.6640
Micro precision	0.6227
Micro recall	0.7111
Hamming loss	0.1359
Training time	6.92 seconds
Inference latency	0.02 ms per example, as recorded

The HCX run demonstrates the performance recorded for its nine-class target space. Its top-3 accuracy is higher than its top-1 accuracy, indicating that the correct target appears among the three highest-ranked candidates more often than it appears as the highest-ranked candidate, under the experiment's metric definition.

The timing values are specific to the supplied run and measurement method. They should not be generalized to different hardware or compared directly with results from other implementations without a controlled timing procedure.

Figure 5: HCX Classification Performance



Explanation: The graph summarizes top-1 accuracy, top-3 accuracy, F1 scores, precision, and empirical conformal coverage for the HCX run.

7.5 Conformal Prediction Results

The supplied HCX experiment records the following conformal prediction measurements.

Measure	Recorded value
Nominal target coverage	90.0%
Empirical coverage	97.4%
Average prediction-set size	3.47
Minimum set size	1
Maximum set size	7
Calibration sample count	1,500
Quantile threshold	0.7196

The recorded empirical coverage exceeds the nominal 90% target for this run. The average prediction-set size of 3.47 indicates that the system returns multiple candidate labels on average under the recorded procedure.

These values should be interpreted as empirical observations from the supplied experiment. They do not establish universal coverage, class-conditional coverage, or robustness under temporal distribution shift. The exact calibration and test split details should be verified before making a formal statistical claim.

7.6 Existing Approaches and HCX-Patent

The following table summarizes the methodological differences between established approaches and HCX-Patent.

Approach	Main method	Primary focus	Relationship to HCX-Patent
PatentBERT	Fine-tuned BERT	Patent text classification	Relevant neural baseline; not re-run in the supplied artifacts
PatentNet	Deep language understanding	Multi-label patent classification	Relevant multi-label comparator; not re-run here
MEXN	Multi-stage extraction	Long patent documents	Relevant document-processing approach; not re-run here
Hierarchical classification methods	Structured label prediction	Parent–child label relationships	Motivates taxonomy-aware evaluation
LIME / SHAP	Feature attribution	Model explanation	Candidate methods for explanation evaluation
Split conformal prediction	Calibration-based prediction sets	Uncertainty estimation	Used in the recorded HCX experiment
HCX-Patent	TF-IDF-based hierarchy-oriented experiment with conformal outputs	Classification, uncertainty, and reproducible research	Preliminary metrics recorded; further validation required

This is a qualitative comparison. It is not a numerical ranking because the methods were evaluated on different datasets, target spaces, and experimental protocols.

7.7 Conventional Versus Proposed Approach

Dimension	Conventional workflow	HCX-Patent approach
Classification	Predicts a class or ranked labels	Records top-1 and top-3 metrics for the HCX run
Taxonomy	May treat labels as flat categories	Treats CPC hierarchy as a research objective; violation rate still needs measurement
Class imbalance	Overall accuracy may conceal rare-class errors	Includes macro-F1 and per-class distribution analysis
Explainability	May provide predictions without validated explanations	Defines a path for attribution and faithfulness evaluation
Uncertainty	May return an uncalibrated confidence score	Records conformal coverage and prediction-set size

Temporal evaluation	Often uses random splitting	Defines chronological evaluation as future work
Reproducibility	Can depend on undocumented settings	Stores experiment metrics and associated artifacts
Comparison validity	May compare unrelated scores	Requires matched data splits for direct model comparison

7.8 Discussion of Results

The baseline experiment shows that classification across 293 classes is challenging under the recorded configuration. Its low macro-F1 indicates that the model performs poorly across the class distribution, and the per-class plot shows why average metrics should be supplemented with class-level analysis.

The HCX run reports stronger metrics on a nine-class task and records conformal prediction coverage above the nominal target. However, the difference between the two runs cannot be attributed solely to the proposed HCX method because the number of samples and target classes differ.

A valid direct comparison requires the baseline and HCX models to use the same records, the same target definitions, the same train/test split, and the same evaluation metrics. The classification task must also be clearly identified as single-label or multi-label.

Future experiments should additionally evaluate hierarchy violations, explanation faithfulness, calibration behavior across rare classes, and performance on later publication years.

7.9 Limitations

The main limitations of the current project are:

1. The baseline and HCX runs use different sample sizes and class spaces.
2. The exact relationship between the nine HCX classes and the 293 baseline classes must be documented.
3. Quantitative hierarchy-consistency measurements are not present in the supplied results.
4. Explanation-faithfulness and stability results are not available in the supplied artifacts.
5. The publication-year graph does not provide measured year-wise model performance.
6. Conformal coverage depends on the validity of the calibration procedure and its assumptions.
7. Reported timing values depend on hardware and measurement methodology.
8. The current results should not be interpreted as proof of state-of-the-art performance.

8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

HCX-Patent is an offline-first patent classification research prototype designed to investigate text classification, taxonomy-aware prediction, explainability, uncertainty estimation, and reproducible evaluation.

The project uses a TF-IDF and Logistic Regression baseline and includes a separate HCX classification experiment. The baseline artifacts describe 9,741 examples and 293 classes, while the HCX experiment reports 15,000 samples and nine target classes. The HCX run records 77.27% top-1 accuracy, 94.53% top-3 accuracy, macro-F1 of 0.6351, micro-F1 of 0.6640, and 97.4% empirical conformal coverage for a nominal target of 90%.

These values provide preliminary evidence about the recorded HCX run. However, because the two runs differ in sample size and target label space, they do not establish a controlled improvement over the baseline.

The project also establishes a methodology for inspecting class imbalance, recording experimental artifacts, and planning further validation. The results highlight the importance of matched evaluation protocols and transparent reporting when developing machine-learning systems for patent classification.

Overall, HCX-Patent provides a foundation for continued research into trustworthy patent classification. Further experiments are required to establish the effectiveness of hierarchy constraints, explanation methods, and temporal robustness.

8.2 Future Scope

Short-Term Improvements

1. Standardize the dataset version, class mapping, and experiment splits.
2. Run baseline and HCX models on identical training and test records.
3. Add confusion matrices and detailed per-class precision, recall, and F1 reports.
4. Validate the prediction-set calculation and calibration procedure.
5. Improve dataset inspection and missing-value reporting.
6. Verify all frontend-to-backend integrations.
7. Record software versions and hardware details for each experiment.

Medium-Term Improvements

1. Integrate a pretrained patent-specific transformer model where computational resources permit.
2. Evaluate hierarchy-aware loss functions or structured prediction methods.

3. Add LIME, SHAP, or other suitable attribution methods.
4. Measure explanation comprehensiveness, sufficiency, and stability.
5. Evaluate conformal coverage and set size across individual classes.
6. Implement chronological evaluation using publication-year splits.
7. Add experiment comparison and model-version management.

Long-Term Improvements

1. Investigate taxonomy-aware classification with measured hierarchy consistency.
2. Study performance under taxonomy updates and distribution shift.
3. Explore long-document encoders for claims and descriptions.
4. Investigate methods for improving performance on rare CPC classes.
5. Evaluate explanation reliability across different model architectures.
6. Develop a more comprehensive research dashboard for comparing models and experimental configurations.
7. Validate the proposed system on additional patent datasets and clearly documented benchmarks.

9. REFERENCES

1. J.-S. Lee and J. Hsiang, "Patent Classification by Fine-Tuning a Pre-Trained BERT Model," 2019. Available: https://arxiv.org/abs/1906.02124
2. A. H. Roudsari et al., "PatentNet: Multi-label classification of patent documents using deep learning based language understanding," *Scientometrics*, vol. 127, pp. 207–231, 2022. Available: https://doi.org/10.1007/s11192-021-04179-4
3. J. Bai, I. Shim, and S. Park, "MEXN: Multi-stage extraction network for patent document classification," *Applied Sciences*, vol. 10, no. 18, Art. no. 6229, 2020. Available: https://doi.org/10.3390/app10186229
4. Y. Zhang et al., "A Hierarchical Feature Extraction Model for Multi-Label Mechanical Patent Classification," *Sustainability*, vol. 10, no. 1, Art. no. 219, 2018. Available: https://doi.org/10.3390/su10010219
5. J. Wehrmann, R. C. A. Pires, and V. A. F. Almeida, "Hierarchical Document Classification as a Sequence Generation Task," in *Proceedings of the ACM/IEEE Joint Conference on Digital Libraries*, 2020. Available: https://doi.org/10.1145/3383583.3398538
6. C. N. Silla Jr. and A. A. Freitas, "A Survey of Hierarchical Classification Across Different Application Domains," *Data Mining and Knowledge Discovery*, vol. 22, pp. 31–72, 2011. Available: https://doi.org/10.1007/s10618-010-0175-9
7. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186. Available: https://doi.org/10.18653/v1/N19-1423
8. I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The Long-Document Transformer," 2020. Available: https://arxiv.org/abs/2004.05150
9. A. Cohan, S. Feldman, I. Beltagy, D. Downey, and D. Weld, "SPECTER: Document-level Representation Learning Using Citation-informed Transformers," in *Proceedings of ACL*, 2020, pp. 2270–2282. Available: https://doi.org/10.18653/v1/2020.acl-main.207
10. M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You?: Explaining the Predictions of Any Classifier," in *Proceedings of ACM SIGKDD*, 2016, pp. 1135–1144. Available: https://doi.org/10.1145/2939672.2939778
11. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. Available: https://papers.nips.cc/paper/7062-a
12. M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic Attribution for Deep Networks," in *Proceedings of the International Conference on Machine Learning*, 2017, pp. 3319–3328.
13. G. Shafer and V. Vovk, "A Tutorial on Conformal Prediction," *Journal of Machine Learning Research*, vol. 9, pp. 371–421, 2008. Available: https://jmlr.org/papers/v9/shafer08a.html
14. A. N. Angelopoulos and S. Bates, "A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification," *Foundations and Trends in Machine Learning*, vol. 16, no. 4, pp. 494–591, 2023. Available: https://doi.org/10.1561/2200000101
15. European Patent Office and United States Patent and Trademark Office, "Cooperative Patent Classification (CPC)." Available: https://www.cooperativepatentclassification.org/
16. Python Software Foundation, "Python Documentation." Available:

https://docs.python.org/3/
17. FastAPI, “FastAPI Documentation.” Available: https://fastapi.tiangolo.com/
18. scikit-learn Developers, “scikit-learn: Machine Learning in Python.” Available: https://scikit-learn.org/stable/
19. React Team, “React Documentation.” Available: https://react.dev/
20. pandas Development Team, “pandas Documentation.” Available: https://pandas.pydata.org/docs/